

RESEARCH ARTICLE

SignFormer-GCN: Continuous sign language translation using spatio-temporal graph convolutional networks

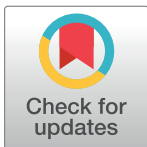
Safaeid Hossain Arib¹*, Rabeya Akter¹*, Sejuti Rahman^{1*}, Shafin Rahman²

1 Department of Robotics and Mechatronics Engineering, University of Dhaka, Dhaka, Bangladesh,

2 Department of Electrical and Computer Engineering, North South University, Dhaka, Bangladesh

* These authors contributed equally to this work.

* sejuti.rahman@du.ac.bd



OPEN ACCESS

Citation: Arib SH, Akter R, Rahman S, Rahman S (2025) SignFormer-GCN: Continuous sign language translation using spatio-temporal graph convolutional networks. PLoS ONE 20(2): e0316298. <https://doi.org/10.1371/journal.pone.0316298>

Editor: Yawen Lu, Purdue University, UNITED STATES OF AMERICA

Received: August 10, 2024

Accepted: December 9, 2024

Published: February 14, 2025

Copyright: © 2025 Arib et al. This is an open access article distributed under the terms of the [Creative Commons Attribution License](https://creativecommons.org/licenses/by/4.0/), which permits unrestricted use, distribution, and reproduction in any medium, provided the original author and source are credited.

Data Availability Statement: We have uploaded the datasets required to replicate our study's findings on sign language translation for three different languages to Kaggle. Each entry below includes the specific translation task, the corresponding dataset name, and a link to access the necessary data: • Bangla Sign Language Translation (BornilDB v1.0 dataset): <https://www.kaggle.com/datasets/rabeyaakter23/bornildb-v1-0-i3d-features-mediapipe-features>. • American Sign Language Translation (How2Sign dataset):

Abstract

Sign language is a complex visual language system that uses hand gestures, facial expressions, and body movements to convey meaning. It is the primary means of communication for millions of deaf and hard-of-hearing individuals worldwide. Tracking physical actions, such as hand movements and arm orientation, alongside expressive actions, including facial expressions, mouth movements, eye movements, eyebrow gestures, head movements, and body postures, using only RGB features can be limiting due to discrepancies in backgrounds and signers across different datasets. Despite this limitation, most Sign Language Translation (SLT) research relies solely on RGB features. We used keypoint features, and RGB features to capture better the pose and configuration of body parts involved in sign language actions and complement the RGB features. Similarly, most works on SLT research have used transformers, which are good at capturing broader, high-level context and focusing on the most relevant video frames. Still, the inherent graph structure associated with sign language is neglected and fails to capture low-level details. To solve this, we used a joint encoding technique using a transformer and STGCN architecture to capture the context of sign language expressions and spatial and temporal dependencies on skeleton graphs. Our method, SignFormer-GCN, achieves competitive performance in RWTH-PHOENIX-2014T, How2Sign, and BornilDB v1.0 datasets experimentally, showcasing its effectiveness in enhancing translation accuracy through different sign languages. The code is available at the following link: <https://github.com/rabeya-akter/SignLanguageTranslation>.

1 Introduction

Millions of individuals who are deaf or hard of hearing communicate through sign languages worldwide. Sign languages, characterized by their unique differences, richness, and complexity, are similar to spoken languages, and they emphasize the significance of developing SLT technology. SLT converts sign language videos to spoken language to ensure effective communication between those who use sign language and those unfamiliar with sign language. This

<https://www.kaggle.com/datasets/rabeyaakter23/how2sign-i3d-features-mediapipe-features>. • German Sign Language Translation (RWTH-PHOENIX-2014T dataset): <https://www.kaggle.com/datasets/rabeyaakter23/rwth-phoenix-2014t-i3d-features-mediapipe-features>. The dataset underlying the results presented in the study are available from: <https://how2sign.github.io/> <https://www.i6.informatik.rwth-aachen.de/~koller/RWTH-PHOENIX/> <https://bornil.bengali.ai/dataset>.

Funding: This research was supported by the Special Innovation Grant from the Ministry of Science and Technology (MoST), Bangladesh, for the fiscal year 2023-2024, Project ID: SRG-232431. This grant is awarded to Dr. Sejuti Rahman. This research also received the Conference Travel and Research Grant (CTRG) for 2023-2024 from North South University, Grant ID: CTRG-23-SEPS-20. The grant recipient is Dr. Shafin Rahman. The funders had no role in study design, data collection and analysis, publication decisions, or manuscript preparation.

Competing interests: The authors have declared that no competing interests exist.

technology has the potential for real-life applications, such as in educational institutions, workplaces, healthcare facilities, public services, etc. Researchers have developed various methodologies to work on both isolated sign language recognition (SLR), which identifies individual signs, and continuous SLT, which translates full sentences and is needed for natural and effective communication [1–17]. However, the state-of-the-art for continuous SLT is still evolving. In this paper, we propose a new method of continuous SLT, especially targeting low-resource languages like Bangla Sign Language (BdSL).

Sign language users employ two distinct types of signals to convey information: manual elements, encompassing hand movement, arm orientation, etc., and non-manual elements, consisting of facial expressions, mouth actions, eye movements, eyebrow gestures, head movements, body postures, etc. [18, 19]. Using only the RGB feature for tracking these signals and training models can be limiting as discrepancies between the backgrounds and signers in training and testing datasets can lead to a decline in translation efficiency [3]. In contrast, the keypoints feature can capture fine-grained details about the pose and configuration of the body parts involved in the sign language action, which is complementary to the RGB feature. Taking motivation from action recognition tasks [20–22], we utilize both RGB and keypoint features to get a better representation of sign language videos. Early research on sign language primarily focused on recognition, but that was limiting effective communication [19, 23–28]. Then, researchers moved their focus working on sign language translation [1–9]. However, very few studies have utilized both RGB and keypoint features simultaneously for SLT [3]. To our knowledge, this is the first work to employ RGB and keypoint features for sign language translation in a gloss-free context.

Existing state-of-the-art methods on SLT have worked with transformer architecture to solve the sequence-to-sequence problem of SLT [5–9]. Transformer architecture can capture the broader context of sign language sequences as it can effectively capture long-distance dependencies, and its attention mechanism focuses on the most relevant video frames while translating texts from sign language videos. Nevertheless, the transformer architecture neglects the inherent graph structure present in the data, which poses challenges in capturing the fine-grained meanings associated with each sign. Graph structures can offer insight into the inter-relationships among the joint points during sign language actions. Consequently, while Transformers are powerful for high-level context understanding, their limitation in capturing low-level details limits their effectiveness in fully translating the complexities of sign language expressions. To solve the problems, we employ a join encoding technique that fuses transformer architecture with Spatial-Temporal Graph Convolutional Networks (STGCN) to get a representation of the sign language videos in both broader and local fine-grained contexts. Moreover, STGCN efficiently learns spatial and temporal dependencies on skeleton graphs.

One major challenge in developing the SLT model is the lack of large and diverse datasets for training. We worked on three sign language datasets: RWTH-PHOENIX-2014T [2], How2Sign [29], and BornilDB v1.0 [30]. Notably, the BornilDB v1.0 [30] dataset presents significant challenges, including only three signers (two males and one female). While developing a model for such low resource SLT, the model tends to learn characteristics of particular individuals, introducing a significant level of bias to the model and thus limiting real-world applications. Additionally, the BornilDB v1.0 [30] dataset is particularly challenging than the previous three datasets because of the varying camera quality, dynamic backgrounds, and inconsistent lighting conditions across videos. To address these challenges in the datasets and to make our model applicable for real-world applications beyond our evaluation datasets, we included both key points and RGB features in our pipeline. We got comparably good results in RWTH-PHOENIX-2014T [2] and How2Sign [29] dataset, and a baseline performance for BornilDB v1.0 [30] dataset. These results demonstrate the effectiveness of our model in the

translation of different sign languages while guiding future research direction for continuous low-resource sign language translation. The primary contributions of the paper can be outlined as:

- **Architectural Fusion:** We integrate both transformer and STGCN architecture to enhance our method's, **SignFormer-GCN**, ability to extract meaningful representation by leveraging contextual and spatio-temporal information at both broader and fine-grained levels.
- **Fusion Strategy:** We explore different fusion strategies between these architectures to identify the most effective one.
- **State-of-the-Art Performance:** SignFormer-GCN provides new state-of-the-art performance of sign language translation in How2Sign [31] and BornilDB v1.0 dataset [30] to guide future research in continuous sign language translation.

The rest of the paper is organized as follows: in Section 2, we survey the previous studies and state-of-the-art on SLT. Then, we introduce our methodology in Section 3. We share our experimental setup and evaluation protocol in the subsequent section. Following this, we report our methodology's quantitative and qualitative results and include a detailed ablation study in Section 4. Finally, we conclude the paper by discussing our findings and the future direction of our work in Section 5.

2 Related work

2.1 Sign language recognition

Initial research on SLR primarily focused on isolated SLR [19, 23–28, 32], which identifies individual signs but lacks the natural flow of sign language communication. Limitations of isolated SLR spurred the development of continuous SLR [10–17], that aims to continuously recognize sign gestures, aligning with sign language grammar and structure and catering to the preferences of sign language users.

2.2 Sign language translation

In pursuit of effective communication between sign language users and those unfamiliar with sign language, extensive research has been done on continuous SLT. Initially, SLT employed Recurrent Neural Networks (RNNs) [33] within the encoder-decoder framework, utilizing either Gated Recurrent Units (GRUs) or Long Short-Term Memory (LSTM) [7, 34–36]. However, addressing the limitations of RNNs in handling long-term dependencies has led to adopting more effective attention-based methods. The transformer network [37], known for its success in various domains [38–43], derives its efficacy from its self-attention mechanism. This feature has been found favourably in recent SLT research, leading to the widespread adoption of transformer architecture. However, transformers can only capture and learn contextual information and patterns. It cannot take advantage of the inherent graph structure of joints in a human body. Graph-based methods have shown significant success in human activity recognition by effectively modeling joint relationships. Spatial-temporal graph convolutional networks' (STGCN) potential to learn spatio-temporal dynamics makes them highly effective at learning and modeling structured patterns from motion sequences. [44, 45]. Consequently, this architecture has been employed to capture spatial and temporal relationships that are essential for SLR [46, 47]. However, a fusion of contextual and spatio-temporal relationships using transformer and STGCN architecture has not been explored, which overlooks important aspects of sign language. In our method, we leveraged both these architectures to get a better and more meaningful representation of the sign gestures.

2.3 Sign language translation based on gloss supervision

SLT based on gloss supervision categorizes current SLT methods into two-stage gloss-supervised methods, end-to-end gloss-supervised methods, and end-to-end gloss-free methods. Gloss, a textual representation of sign gestures in spoken or written language, serves as an intermediary in the first two approaches [1–4]. However, acquiring gloss annotations can be costly as it requires the expertise of sign language professionals. In contrast, gloss-free methods do not rely on intermediary gloss annotations, directly translating sign language videos into spoken language texts [5–9]. Our method belongs to the gloss-free category as we utilize a direct translation from sign language video to spoken language text without any involvement of gloss annotation.

2.4 Tokenization methods of sign language videos

Various SLT approaches have employed different tokenization methods for sign language videos. Some utilized 2D CNN features extracted from video frames at the gloss-level [1, 2]. Inflated 3D convnets (I3D), initially designed for action recognition [48], have been further trained using sign language data [7, 8, 49–52]. S3D [53] features have been employed after pre-training with the WLASL dataset and kinetics [54]. Additionally, some approaches have used pose estimators [55, 56] to represent video sequences, as they provide information on the position and movement of body parts [6, 35, 36, 57]. Lastly, some methods combined video and keypoint features to capture more meaningful representation [3, 58]. Our method aligns with this last-mentioned method as we use both RGB video encoding and keypoint encoding.

3 Methodology

In this section, we introduce SignFormer-GCN, which is an end-to-end model that learns to translate sign language video sequences into spoken language directly.

Problem Formulation: Given a sign language video $V_i = \{f_1, f_2, \dots, f_t\}$ with t frames, our goal is to learn the conditional probabilities $P(S_i|V_i)$ of generating a translated sentence $S_i = \{w_1, w_2, \dots, w_n\}$ with n words. The training dataset consists of a set of tuples $\{(V_i, S_i); i \in [1, M]\}$ where M represents the total number of training videos.

Solving the SLT problem presents quite a few challenges. Sign gestures can vary in length because different signers do them at different paces. Besides, as each video frame does not map one to one with each of the tokens of translated sentences, the detailed meaning expressed through sign gestures might include subtle local aspects because of differences in grammatical rules and ordering in sign languages and spoken languages. Because of transformer networks' effectiveness in modelling sequence-to-sequence tasks, recent methods on SLT employed this architecture [2]. However, it is important to recognize that the representation found in this architecture has its limitations. While transformer architecture excels at learning contextual relationships, this architecture struggles to capture the topological aspect of the joints of a human body, which is equally important in sign language.

Solution strategy: Sign language gestures that involve movements of human body joints are best represented through a spatio-temporal skeleton graph. We can use such a graph to learn a new set of feature representations. To bridge the limitation of transformer architecture, we employ Spatio-Temporal Graph Convolutional Networks (STGCN) to extract the relationship between spatial and temporal features from the skeletal structure. Using only one type of feature representation from the architecture is limiting as they can give both contextual and spatio-temporal information together. To make the translation task more efficient, we incorporated transformer and STGCN architecture in our method to learn a better and more meaningful representation.

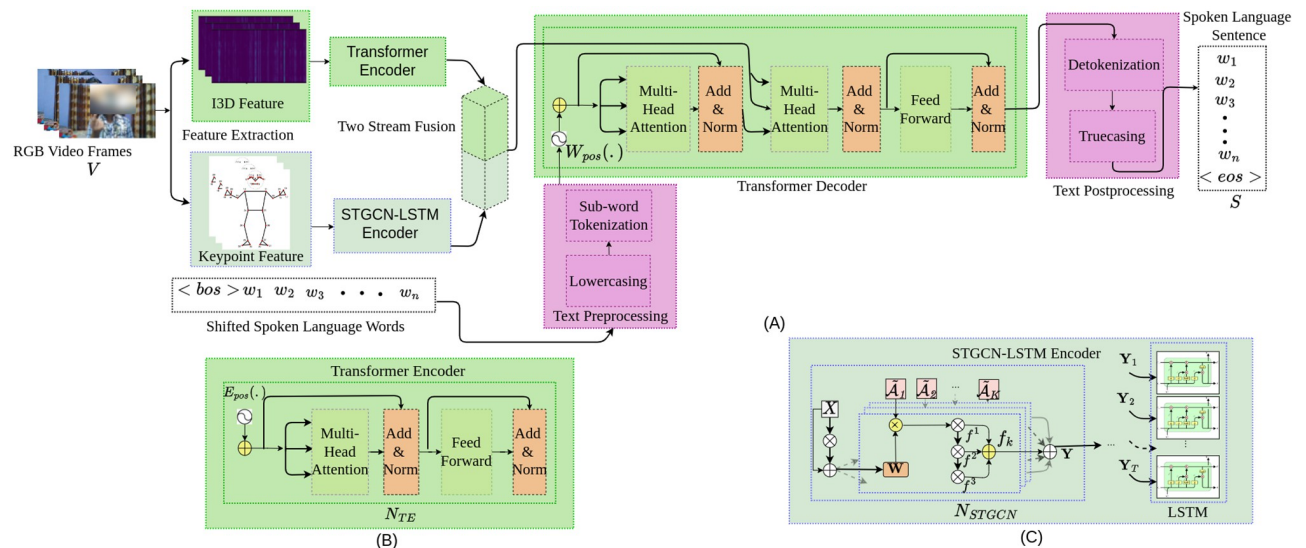


Fig 1. Overview of the SignFormer-GCN methodology for sign language translation. (A) An overview of our two-stream fusion methodology, SignFormer-GCN, for sign language translation. The methodology fused the RGB feature processed by the transformer encoder, and the STGCN-LSTM encoder processed the keypoint feature, and the fused output was passed to a transformer decoder for final translation. (B) An overview of the transformer encoder. (C) An overview of STGCN-LSTM encoder.

<https://doi.org/10.1371/journal.pone.0316298.g001>

3.1 Model overview

An overview of our model is shown in Fig 1. We fuse two stream encoding processes to encode the sign language videos, $V \in \mathbb{R}^{T \times N \times C}$. In the first stream, i3d features, $E_t \in \mathbb{R}^{T \times C}$, are extracted using an i3d network. Then, this feature is positionally encoded before being fed into a transformer encoder that is formed with several transformer encoder blocks, N_{TE} . The ultimate output encoding of this stream is denoted as $Z_t \in \mathbb{R}^{T \times C}$. In the second stream, we extract keypoint features using the Mediapipe algorithm. This process establishes a spatio-temporal graph structure, $G = (V, E, A)$, incorporating K joints and T frames. Then, the keypoint data is passed through an STGCN-LSTM Encoder formed by multiple STGCN blocks, N_{STGCN} , followed by an LSTM layer. This stream finally gives an encoding represented as $Y \in \mathbb{R}^{T \times N \times (C_{out})}$. These two streams are fused in the next step to pass the final encoding to a transformer decoder formed with multiple transformer decoder blocks, N_{TD} . The preprocessed text is also passed to the transformer decoder. The decoder generates tokenized output, which is detokenized in the post-processing step to generate the predicted spoken language text, $S \in \mathbb{R}^N$. Commonly used notations are summarized in Table 1 for clarity.

3.2 Joint encoding

Video encoding. We employ an I3D network [59] over 16-frame video clips, $F_{I3D}(V_{t:t+15})$, to give an embedding, ψ_v . This number of frames is taken because of its effectiveness in sign language recognition methods [24, 49]. Then, we temporarily aggregate the ψ_v to a vector of constant size. Finally, the vector is transformed into the C dimensional embedding space.

$$E_t = P(F_{\text{temporal}}(F_{I3D}(V_{t:t+15}))) \quad (1)$$

Here, $E_t \in \mathbb{R}^C$ is the feature embedding of the video, $F_{\text{temporal}}(\cdot)$ is the temporal aggregation function and $P(\cdot)$ is the projection operation. As for translating sign language, the position of

Table 1. Commonly used notations.

Notations	Descriptions
$F_{\text{temporal}}(\cdot)$	Temporal aggregation function
$P(\cdot)$	Projection operation
$E_t \in \mathbb{R}^C$	Feature embedding of the video
$\hat{\mu} \in \mathbb{R}^C$	Embedded representation of the text
$E_{\text{pos}}(\cdot)$	Positional encoding function for the text
G	Graph.
$\mathcal{V}$	Set of nodes in a graph, $V = \{v_{ij} t = 1, \dots, T; j = 1, \dots, J\}$
$\mathcal{E}$	Set of edges in a graph, $E = \{v_{ij}v_{ik} \cup v_{ij}v_{(t+1)j} t = 1, \dots, T, (j, k) \in S\}$
$\mathcal{A}$	Adjacency matrix
$\mathcal{A}_k$	k th adjacency matrix
$\mathcal{I}$	Identity matrix
D_k	k th diagonal degree matrix
N	Number of nodes, $N = \mathcal{V} $
J	Number of joints
T	Number of frames
S	Connectivity of human skeleton structure
l	The layer index
t	The time step index
$\otimes$	Temporal convolution operation.
$\oplus$	Concatenation operation
C_m	Number of input channels
C_n	Number of output channels of STGCN
C_o	Number of output channels of LSTM
$\mathbf{X} \in \mathbb{R}^{T \times N \times C_m}$	Input feature matrix of STGCN
$\mathbf{Y}^l \in \mathbb{R}^{T \times N \times C_n}$	Output from the STGCN block
Γ_i^l	The i^{th} temporal kernel of l^{th} STGCN block
$\mathbf{Y}_r \in \mathbb{R}^{N \times NC_n}$	Input feature of LSTM
$\sigma_s(\cdot)$	Sigmoid activation function
$\sigma_c(\cdot)$	Tanh activation function.
W_i, W_o, W_f, W_c	Weight matrices for the input features of LSTM
U_i, U_o, U_f, U_c	Weight matrices for the hidden states of LSTM
$b_f, b_i, b_o, b_c \in \mathbb{R}^{C_m}$	Bias vectors of LSTM

<https://doi.org/10.1371/journal.pone.0316298.t001>

sign gestures in the entire video sequence is essential information. However, transformer networks do not have access to positional information. To overcome this limitation, we introduce temporal ordering information to the embedded representations of the video using positional encoding $E_{\text{pos}}(\cdot)$.

$$\hat{E}_t = E_t + E_{\text{pos}}(t) \quad (2)$$

We train a transformer encoder model using the positional encoded embedding of the video frames, $\hat{E}_{1:T}$. Initially, the embedding of the video frames is passed to a self-attention layer, which is responsible for learning the contextual connections among the embedding of the video frames. Then, the results of the self-attention layers are forwarded through a non-linear point-wise feed-forward layer. Throughout the process, residual connections and normalization are applied for effective training. The process can be formulated using the following

equation, where Z_t is the spatiotemporal representation of the frame f_t at time step t , given the embedding of all the frames of the video, $\hat{E}_{1:T}$. Multiple encoder blocks are stacked to extract the features.

$$Z_t = \text{TransformerEncoder}(\hat{E}_t | \hat{E}_{1:T}) \quad (3)$$

Keypoint encoding. We employ Mediapipe [56] algorithm for extracting keypoint features. Then, we construct an undirected spatiotemporal graph structure $G = (V, E, A)$ with J joints and T frames, comprising both intra-body and frame-by-frame connections. The set of vertices $V = \{v_{tj} | t = 1, \dots, T; j = 1, \dots, J\}$ includes all the joints in the sequence of frames. The feature vectors of each vertex v_{tj} will be 3D coordinate vectors represented by $M_g(v_{tj}) = \{x, y, z\}$, where x, y , and z are coordinates along each axis in a 3D coordinate space. The joints are connected with edges, $E = \{v_{tj}v_{tk} \cup v_{tj}v_{(t+1)j} | t = 1, \dots, T, (j, k) \in S\}$, according to the connectivity of the human skeletal structure, S , for a frame, and each of the joints connection to itself through consecutive frames. $A \in \mathbb{R}^{N \times N}$ is the adjacency matrix of the graph G . The adjacency matrix element is 0 if there is no connection between joints and one if there is a connection between joints.

We pass the keypoint data of a video, $X \in \mathbb{R}^{T \times N \times C_m}$, to STGCN block to extract features from the body-joints. Here, T denotes the temporal length; N denotes the number of skeleton joints; C_m denotes the number of input channels. This at first extracts the spatial features, $Z \in \mathbb{R}^{T \times N \times (C_m + C_{filter})}$, by performing temporal convolution to the input keypoint sequences using kernel, Γ and concatenating with input to extract spatial features. Here, C_{filter} is the number of filters employed in temporal convolution.

$$Z = X \oplus (\Gamma \otimes X) \quad (4)$$

Here, $\otimes$ and $\oplus$ denote the temporal convolution and concatenation respectively. We perform graph convolution on Z at time t using the layer, l , wise update rule of GCN [60].

$$Z_t^{l+1} = \sigma \left(\sum_{k=0}^K \tilde{D}_k^{-\frac{1}{2}} (A_k + I) D_k^{-\frac{1}{2}} Z_t^l W_k^l \right) \quad (5)$$

Here, k is the number of level of aggregation, A_k is the k th adjacency matrix, I is the identity matrix, D_k is a diagonal degree matrix, W_k is the learnable weight matrix and σ is an activation function.

Then, three layers of Temporal Convolution Layers (TCNs) with the same padding and kernel Γ^1, Γ^2 , and Γ^3 , respectively, were implemented to obtain temporal features at various levels. We combined both low-level and high-level features to identify movement patterns while performing a gesture in sign language at various levels of complexity. The operations can be described as the following equations.

Here, spatiotemporal features extracted from the k th layer are represented by f_k . Lastly, the output from each layer is concatenated to get the final output.

$$Y = f_1 \oplus f_2 \oplus \dots \oplus f_k \quad (6)$$

The result of the STGCN block is denoted as $Y \in \mathbb{R}^{T \times N \times C_n}$. Here, $C_n = \sum_{i=1}^k C_i^l$ refers to a tensors of three dimensions. We stack multiple STGCN blocks in our method to extract complex spatio-temporal features. After that, we pass the output of the last STGCN block, Y^l , to the LSTM block. The LSTM block captures the sequential dependencies in spatio-temporal feature vectors. We used the LSTM block after the STGCN blocks because of its ability to consider the

change of spatiotemporal features along the temporal dimension. LSTM is very helpful in the processing of variable-length sequences. In sign language, the same sign done by different signers can have different lengths because it is done at different paces and in different situations. For this, LSTM is a better fit for this sequence problem solution.

For this, Y^l were reshaped to $Y^r \in \mathbb{R}^{T \times NC_n}$, which is the input to the LSTM. LSTM involves the following operations:

Here, i_t^t, f_t^t and $o_t^t \in \mathbb{R}^{C_m \times C_o}$ denote the input gate, forget gate and output gate respectively. C_o is the number of units in an LSTM cell. W_f, W_i, W_o and $W_c \in \mathbb{R}^{C_m \times NC_n}$ are the weight matrices, U_f, U_i, U_o and $U_c \in \mathbb{R}^{C_m \times C_m}$ are the weight matrices for the preceding hidden state, b_f, b_i, b_o and $b_c \in \mathbb{R}^{C_m}$ are four bias vectors, σ_g and σ_c are the sigmoid and tanh activation functions. The cell state and the hidden states are denoted by C^t and $h^t \in \mathbb{R}^{C_m \times C_o}$ respectively.

Two-stream fusion module. The fusion process integrates the I3D feature obtained from the transformer encoder, denoted as Z_t , and the encoding from the STGCN-LSTM architecture, denoted as L_t . The fused encoding, denoted as Fused Encoding, is mathematically represented as:

$$\text{Fused Encoding} = Z_t + L_t \quad (7)$$

3.3 Training pipeline

Text encoding. For text encoding, we used Sentencepiece tokenizer [61], which segments the text into sub-word units. Splitting longer sentences into sub-word units enables us to learn better representations of phonetic variants and compound words. This approach is beneficial for acquiring less common words and addresses issues related to out-of-vocabulary words. After tokenization, positional encoding is done to have temporal information of the text. Following is the formulation of this process, where $\hat{\mu} \in \mathbb{R}^C$ is the embedded representation of the text, and $E_{pos}(\cdot)$ is the positional encoding function for the text.

$$\hat{\mu} = \text{SubWordEmbedding}(w_n) + W_{pos}(n) \quad (8)$$

Decoding output. At the beginning of the target spoken language sentence, we add a special token $\langle bos \rangle$. Then, position-encoded word embedding is fed to a masked self-attention layer. Masking ensures that each token only considers its preceding tokens while gathering contextual information. Masking is necessary as the decoder cannot access future output tokens during inference. The combined video and text embedding are passed to the decoder layer to learn the reference and target sequence mapping. The transformer decoder is trained to generate target words sequentially until it reaches the $\langle eos \rangle$ token. The decoding process is formulated as follows:

$$H_{n+1} = \text{TransformerDecoder}(\hat{\mu}_u | \hat{\mu}_{1:n-1}, Z_{1:T}) \quad (9)$$

The overall probability of the sentence, $p(S|V)$ is calculated by multiplying the probabilities of individual words given their respective contexts, H_u . This can be formulated as:

$$p(S|V) = \prod_{u=1}^n p(w_u | H_u) \quad (10)$$

Loss optimization. We employed Labeled Smoothed Cross Entropy ($\mathcal{L}_{CE}$) loss to train our method [31, 62, 63]. This loss is a modified version of cross entropy with the integration of label smoothing, encouraging the model to generalize translation to unseen translation. This loss function clusters the representations for training examples of the same class. $\mathcal{L}_{CE}$ loss can be formulated by the following equation, where $ce(x)$ denotes the standard cross-entropy loss of x , ϵ is a small positive number, y is the correct class, $\hat{y}$ is the incorrect class, and N is the number of classes.

$$\mathcal{L}_{CE} = (1 - \epsilon)ce(y) + \epsilon \sum \frac{ce(\hat{y})}{N} \quad (11)$$

3.4 Analysis

Most works in SLT utilize transformer architecture to address the problem [31]. While transformer architecture excels at sequence-based tasks like SLT due to its ability to understand broader context and long-range dependencies, it often overlooks the inherent graph structure in sign language videos. In their experiment, [31] observed that when working with poses, the decoder tends to disregard the conditioning provided by the encoder and operate primarily as a language model. This behaviour could be attributed to feeding poses as sequences of one-dimensional arrays containing only landmark coordinates. Consequently, they excluded keypoint features from their architecture, ignoring pose's graph-like structure. Our approach, however, incorporates additional streams of keypoint features using an STGCN-LSTM encoder alongside the RGB feature and transformer architecture stream. This allows our model to benefit from a richer input representation, capturing both visual and pose-related details. As a result, this enhancement can improve the model's understanding of sign language videos and potentially increase translation quality.

Most of the works in SLT that utilized graph-based methods focus on SLR task [64–68]. Among the few works that address SLT using graph-based methods, they incorporate gloss annotation in their training pipeline [69].

Among the works in SLT that utilize both RGB and keypoint features, Chen et al. [3] employ gloss annotation to train their model, which is restricting due to the expensive nature of gloss annotation. In contrast, our approach does not rely on gloss to train the model. We combine RGB and keypoint features to extract two distinct latent representations, capturing contextual and spatio-temporal information. Furthermore, Chen et al. [3] use keypoints as heatmaps, which has potential disadvantages. Using keypoints as heatmaps can result in the loss of fine-grained spatial information. When keypoints are represented as heatmaps, the exact spatial coordinates of keypoints may not be preserved with the same level of precision as when using raw keypoints coordinates. This can impact the model's ability to capture subtle movements and gestures in sign language, which are crucial for accurate translation. In contrast, our method preserves this fine-grained spatial information by using raw keypoint coordinates, potentially leading to more accurate capture and interpretation of sign language gestures.

4 Experiments

4.1 Datasets

We experimented with SignFormer-GCN on three publicly available datasets: RWTH-PHOENIX-2014T [2], How2Sign [29], and BorniDB v1.0 [30]. A comparative analysis of these datasets is presented in Table 2.

Table 2. Comparative analysis of SLT datasets based on language, number of signers, video duration (in hours), vocabulary size (in thousands), and domain attributes.

Dataset	Language	Signers	Duration(h)			Vocabulary(k)			Domain
			Train	Val	Test	Train	Val	Test	
RWTH-PHOENIX-2014T [1]	DGS	9	9.2	0.6	0.7	2	0.9	1	Weather Forecast
How2Sign [29]	ASL	11	69.6	3.9	5.6	15.6	3.2	3.6	Instructional
BornilDB v1.0 [30]	BdSL	3	49.86	5.56	9.86	14.1	3.9	5.4	Not Specified

<https://doi.org/10.1371/journal.pone.0316298.t002>

- RWTH-PHOENIX-2014T [1] is the most used dataset for SLT assessment. The dataset contains German Sign Language (DGS) videos on weather forecasts collected from the German public television station PHOENIX. Along with videos, the dataset has text and gloss annotations in German.
- Compared to the former two datasets, How2Sign [29] is a much larger and much complex dataset. The dataset is made up of American Sign Language(ASL) videos and text annotations of the videos. The dataset is on Instructional Videos on ten different topics.
- BornilDB v1.0 Dataset [30] contains sign language videos of Bangla Sign Language(BdSL) and translated text annotations in Bangla language. Three signers performed the sign language on different topics, but no specific topic is outlined in the dataset paper.

4.2 Evaluation protocol

We use reduced BLEU (rBLEU) [50] and BLEU-n [70] scores to measure the performance of our method. BLEU-n score calculates the similarity of the predicted translation to the reference translation. It is compared using n-grams. The precision of each n-gram is calculated by counting the matching n-gram between the reference translation and the predicted translation. The precision is uniformly weighted from 1-grams to n-grams. BLEU score is computed using the following formula:

$$\text{BLEU} - n = \text{BP} \times \exp\left(\frac{1}{n} \sum_{i=1}^n \log(p_i)\right) \quad (12)$$

$$\text{BP} = \begin{cases} 1 & \text{if } \text{MT}_{\text{length}} > \text{Ref}_{\text{length}} \\ \exp\left(1 - \frac{\text{Ref}_{\text{length}}}{\text{MT}_{\text{length}}}\right) & \text{if } \text{MT}_{\text{length}} \leq \text{Ref}_{\text{length}} \end{cases} \quad (13)$$

Here, utilizing the “Brevity Penalty,” BP, longer sentences are encouraged, and p_i is the precision for n-grams of length i. $\text{MT}_{\text{length}}$ represents the length of the machine translation, and $\text{Ref}_{\text{length}}$ denotes the length of the reference translation. Higher BLEU-n scores indicate better translation performance.

rBLEU score is the computation of the BLEU score after removing certain words from the ground truth and prediction. These certain words are generated before training of the model as listed words. Even though listed words are used in the training data, they do not contribute much to the meaning of the sentences. A higher rBLEU score indicates better translation performance.

Table 3. Optimal hyper-parameters are discovered during experiments.

Hyperparameters	Values
Text Processing	Yes
Batch Size	32
Layers of Encoder	6
Layers of Decoder	3
Attention Heads	4
Learning Rate	1e-3
Feed-Forward Network Dimension	1024
Embedding Dimension	256
Learning Rate Scheduler	cosine

<https://doi.org/10.1371/journal.pone.0316298.t003>

4.3 Implementation details

In the training phase of our model, we employed a batch size of 32 and conducted 250 epochs, each requiring approximately 2.5 minutes for processing on a single NVIDIA GeForce RTX 3090 GPU. We validated after every two epochs of training. Other hyperparameters values are listed in Table 3.

4.4 Main results

Quantitative results. The effectiveness of our model on the How2Sign dataset [29], compared to the available baseline for this dataset, is illustrated in Table 4.

In Table 5, we present the comparative results, which include BLEU scores of our model in contrast to the comparison models on the RWTH-PHOENIX-2014T dataset [1]. The results for Joint-SLT [2] were obtained from [72], where they reproduced the results in a gloss-free context. At the same time, we relied on the results originally reported in their respective papers for other models. As illustrated in Table 5, our model significantly improves translation performance while being highly efficient in terms of computational resources. Specifically, our model has only 9.43 million parameters, compared to 115.41 million parameters in the GFSLT-VLP [62] method. Additionally, the training time for our model is approximately 6 hours (200 epochs) on a single NVIDIA GeForce RTX 3090 GPU; in comparison, the GFSLT-VLP [62] approach requires 60 hours across two training phases, using four NVIDIA RTX 3090 GPUs. The GFSLT-VLP [62] approach relies on a symmetric cross-entropy loss similar to the one used in CLIP [73], which is very resource-intensive compared to our method. Regarding inference speed, our model achieves a response time of 0.03–0.034 seconds, while the GFSLT-VLP [62] model takes 0.8–1 second. Our model achieves strong translation performance while being significantly more efficient regarding training time, model size, and inference speed.

In the absence of an established baseline for the BorniDB v1.0 dataset [30], we benchmarked on the dataset in Table 6 to guide future research in BdSL.

Table 4. Performance comparison of gloss-free translation methods on the How2Sign dataset. (Bold result indicates SignFormer-GCN's result).

	Validation					Test				
	rBLEU	BLEU-1	BLEU-2	BLEU-3	BLEU-4	rBLEU	BLEU-1	BLEU-2	BLEU-3	BLEU-4
slt_how2sign [31]	2.79	35.2	20.62	13.25	8.89	2.21	34.01	19.3	12.18	8.03
asl_video2text [71]	3.29	35.25	21.03	13.76	9.39	2.56	33.20	19.12	12.10	7.95
SignFormer-GCN	3.97	37.37	22.18	14.45	9.9	2.96	34.91	19.98	12.72	8.53

<https://doi.org/10.1371/journal.pone.0316298.t004>

Table 5. Performance comparison of gloss-free translation methods in the RWTH-PHOENIX-2014T dataset.

Methods	BLEU-1	BLEU-2	BLEU-3	BLEU-4
Conv2d-RNN [1]	27.10	15.61	10.82	8.35
+ Luong Attn. [1] + [74]	29.86	17.52	11.96	9.00
+ Bahdanau Attn. [1] + [75]	32.24	19.03	12.83	9.58
Joint-SLT [2]	30.88	18.57	13.12	10.19
Tokenization-slt [7]	37.22	23.88	17.08	13.25
TSPNet-Sequential [52]	35.65	22.80	16.60	12.97
TSPNet-Joint [52]	36.10	23.12	16.88	13.41
GASLT [72]	39.07	26.74	21.86	15.74
GFSLT-VLP [62]	43.71	33.18	26.11	21.44
SignFormer-GCN	41.19	30.89	24.23	19.75

<https://doi.org/10.1371/journal.pone.0316298.t005>

Table 6. Performance of our gloss-free translation method on the BornilDB v1.0 dataset.

	Validation				Test			
	BLEU-1	BLEU-2	BLEU-3	BLEU-4	BLEU-1	BLEU-2	BLEU-3	BLEU-4
SignFormer-GCN	7.62	3.05	1.37	0.72	7.37	2.89	1.18	0.58

<https://doi.org/10.1371/journal.pone.0316298.t006>

Qualitative results. The qualitative results of our methodology on all three datasets are reported in this section.

The qualitative performance of our model on the How2Sign [51] dataset is shown in Table 7. In Table 8, the generated translation of RWTH-PHOENIX-2014T [1] dataset is shown using our best-performing model. As the reference and generated translation are in German, the English translation is also provided for a better understanding of the result. Table 9 displays the translated results of the BornilDB v1 [30] dataset using our most successful model. Given that the reference and generated translations are in Bangla text, English translations are also included in the table.

4.5 Ablation study

We conducted our ablation investigation on the RWTH-PHOENIX-2014T [1] dataset to enhance the structure and pinpoint the model that performs optimally.

Table 7. Reference and prediction of How2Sign [51] dataset with English translation.

No.	Type	Translation
1	Reference:	one more time.
	Prediction:	one more time.
2	Reference:	the other thing i did was i had a little bit of line out.
	Prediction:	another thing i would do is i have a little bit of line out.
4	Reference:	and a very natural enhanced lip.
	Prediction:	and very natural lips.
5	Reference:	just enough to build up some chest strength.
	Prediction:	enough to build some chest strength.
6	Reference:	going to walk on to our next step.
	Prediction:	and we're going to walk on our next step.
8	Reference:	keep your eyes positioned up.
	Prediction:	so keep your eye position up.

<https://doi.org/10.1371/journal.pone.0316298.t007>

Table 8. Reference and prediction of RWTH-PHOENIX-2014T [1] dataset with English translation.

No.	Type	German (English Translation)
1	Reference:	guten abend liebe zuschauer (good evening dear viewers)
	Prediction:	guten abend liebe zuschauer (good evening dear viewers)
2	Reference:	ihnen noch einen schönen abend (Have a nice evening)
	Prediction:	ihnen noch einen schönen abend (Have a nice evening)
3	Reference:	hallo und guten abend (Hello and good evening)
	Prediction:	hallo und guten abend (Hello and good evening)
4	Reference:	und nun die wettervorhersage für morgen diensttag den fünfundzwanzigsten mai (And now the weather forecast for tomorrow, Tuesday May 25th)
	Prediction:	und nun die wettervorhersage für morgen diensttag den fünfundzwanzigsten mai (And now the weather forecast for tomorrow, Tuesday May 25th)

<https://doi.org/10.1371/journal.pone.0316298.t008>

Effects of varying the number of STGCN layers. Our experiment investigates the effect of having a different number of STGCN layers. While increasing the number of STGCN layers lets our method acquire a better and more complex representation, it also exposes the method to a higher risk of overfitting. With this objective in mind, we train our method using one to six STGCN layers. As seen in Table 10, the ability of our method to translate gets better with additional STGCN layers initially. However, as we continued to add more STGCN layers, the

Table 9. Reference and prediction sentences of BornilDB v1 [30] dataset with English translation.

No.	Type	Bangla (English Translation)
1	Reference:	সব কিছু ঠিক আছে তো? (Is everything okay?)
	Prediction:	সব ঠিক আছে? (Is everything fine?)
2	Reference:	না, আমার মনে হয় না। (No, I don't think so.)
	Prediction:	না, আমার মনে হয় নেই। (No, I don't feel like it.)
3	Reference:	আমি ভালো, বেশ ভালো। (I'm good, pretty good.)
	Prediction:	আমি তো অনেক ভাল আছি। (I am very well.)
4	Reference:	আমায় ছেড়ে দাও! (Leave me alone!)
	Prediction:	আমাকে ছেড়ে দাও। (Let me go.)
5	Reference:	ও এসে গেছে! (He has come!)
	Prediction:	সে এসে গেল। (He has come.)
6	Reference:	সে জানত না? (Didn't he know?)
	Prediction:	সে জানে না? (Does he not know?)

<https://doi.org/10.1371/journal.pone.0316298.t009>

Table 10. BLEU1, BLEU2, BLEU3 and BLEU4 scores for different STGCN layer configurations.

#Layer	Validation				Test			
	BLEU1	BLEU2	BLEU3	BLEU4	BLEU1	BLEU2	BLEU3	BLEU4
1	40.88	30.40	23.86	19.57	41.02	30.03	23.25	18.87
2	40.99	30.41	23.96	19.72	41.05	30.27	23.62	19.26
3	40.97	30.26	23.6	19.23	41.19	30.89	24.23	19.75
4	41.40	30.61	23.66	19.19	40.99	30.15	23.45	19.06
5	40.09	29.75	23.27	18.95	41.22	30.45	23.64	19.16

<https://doi.org/10.1371/journal.pone.0316298.t010>

method overfits the training data, leading to a performance decrease in the test set. For this reason, we use three layers of STGCN in our best-performing model.

Effects of varying the number of transformer encoder and decoder layers. In our next experiment, we explore the impact of employing varying numbers of encoder and decoder layers in the Transformer architecture to find the optimum ones. While increasing the number of layers of encoder and decoder, our method acquires better and more complex representation, and it also exposes the method to a higher risk of overfitting. Considering this objective, we train our method using different encoder and decoder layer combinations. As seen in Table 11, the ability of our method to translate improves with additional encoder and decoder layers. Our model performs best with six layers of encoder layer and three layers of decoder layer.

Effects of varying the number of LSTM layers. As seen in Table 12, the ability to translate our method improves initially with additional LSTM layers. But, as we continued to add more LSTM layers, the method overfits the training data, leading to a performance decrease in the test set. For this reason, we use one layer LSTM in our best-performing model.

Effectiveness of different fusion strategy. To find the best fusion strategy, we experimented with several different ones shown in Table 13. In this work, three different kinds of fusion techniques were used. These fusion solutions combined the encoding of the transformer encoder and the STGCN-LSTM encoder, two distinct architectural streams. Three fusion procedures were used: Fusing with Summation, Fusing with a Linear Layer, and Fusion Using an LSTM Layer. Our technique included Fused with Summation because it outperforms the other strategies.

Table 11. BLEU1, BLEU2, BLEU3 and BLEU4 scores for different encoder and decoder layer configurations.

# Encoder Layer	# Decoder Layer	Validation				Test			
		BLEU1	BLEU2	BLEU3	BLEU4	BLEU1	BLEU2	BLEU3	BLEU4
6	3	40.97	30.26	23.6	19.23	41.19	30.89	24.23	19.75
2	2	39.15	28.43	21.91	17.7	40.45	29.55	22.78	18.27
3	3	41.22	30.29	23.55	19.16	40.69	30.23	23.53	19.1

<https://doi.org/10.1371/journal.pone.0316298.t011>

Table 12. BLEU1, BLEU2, BLEU3 and BLEU4 scores for different LSTM layer configurations.

#Layer	Validation				Test			
	BLEU1	BLEU2	BLEU3	BLEU4	BLEU1	BLEU2	BLEU3	BLEU4
1	40.97	30.26	23.6	19.23	41.19	30.89	24.23	19.75
2	41.19	30.72	23.98	19.50	41.17	30.55	23.75	19.31
3	40.87	30.19	23.32	18.84	41.58	30.59	23.52	18.89

<https://doi.org/10.1371/journal.pone.0316298.t012>

Table 13. rBLEU, BLEU1, BLEU2, BLEU3 and BLEU4 scores for different fusion strategy.

Fusion Strategy	Validation					Test				
	rBLEU	BLEU1	BLEU2	BLEU3	BLEU4	rBLEU	BLEU1	BLEU2	BLEU3	BLEU4
LSTM layer	8.51	38.33	27.62	21.11	16.96	6.59	37.96	28.05	21.72	17.66
Linear layer	8.29	40.13	29.04	22.25	17.9	5.97	40.5	29.54	22.67	18.34
Summation	8.76	40.64	29.8	23.27	19.01	6.57	40.28	29.32	22.48	18.02

<https://doi.org/10.1371/journal.pone.0316298.t013>

Table 14. rBLEU, BLEU1, BLEU2, BLEU3 and BLEU4 scores for excluding and including STGCN-LSTM encoder.

Fusion Strategy	Validation					Test				
	rBLEU	BLEU1	BLEU2	BLEU3	BLEU4	rBLEU	BLEU1	BLEU2	BLEU3	BLEU4
Excluding STGCN-LSTM	9.56	40.68	29.76	22.95	18.63	6.40	41.23	30.39	23.46	18.80
Including STGCN-LSTM	9.61	40.98	30.27	23.60	19.24	8.65	41.19	30.89	24.33	19.75

<https://doi.org/10.1371/journal.pone.0316298.t014>

Effectiveness of STGCN-LSTM encoder. We trained our model using and excluding the encoder to find the effectiveness of using the STGCN-LSTM encoder in our architecture. As shown in Table 14, including the STGCN-LSTM encoder in the architecture improves the translation performance.

4.6 Limitations

Our approach used transformer architecture, which is resource-intensive due to high computational requirements. Besides, to obtain SLT models for different sign languages, separate training with distinct datasets using the same model is required. This approach limits the model's ability to translate across multiple languages simultaneously.

5 Conclusion and future direction

In this paper, we introduced an approach that uses video and keypoint encoding through the transformer and STGCN architecture to find meaningful contextual and spatiotemporal representation at broader and fine-grained levels to translate sign language videos better. We evaluated our approach on three sign language datasets of three sign languages and reported comparatively good performance for How2Sign [29], RWTH-PHOENIX-2014T [1] and Bor-nilDB v1.0 [30] dataset. In future work, we would like to expand our approach to learn better representation from sign language video by reducing the semantic gap between video encoding, keypoint encoding, and text encoding.

Author Contributions

Conceptualization: Sejuti Rahman, Shafin Rahman.

Formal analysis: Safaeid Hossain Arib, Sejuti Rahman.

Funding acquisition: Sejuti Rahman.

Methodology: Safaeid Hossain Arib, Rabeya Akter.

Resources: Sejuti Rahman.

Supervision: Sejuti Rahman, Shafin Rahman.

Validation: Safaeid Hossain Arib, Rabeya Akter.

Visualization: Rabeya Akter.

Writing – original draft: Safaeid Hossain Arib, Rabeya Akter.

Writing – review & editing: Sejuti Rahman, Shafin Rahman.

References

1. Camgoz NC, Hadfield S, Koller O, Ney H, Bowden R. Neural sign language translation. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. IEEE; 2018. p. 7784–7793.
2. Camgoz NC, Koller O, Hadfield S, Bowden R. Sign language transformers: Joint end-to-end sign language recognition and translation. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR); 2020. p. 1682–1691.
3. Chen Y, Zuo R, Wei F, Wu Y, Liu S, Mak B. Two-stream network for sign language recognition and translation. *Advances in Neural Information Processing Systems*. 2022; 35:17043–17056.
4. Zhou H, Zhou W, Qi W, Pu J, Li H. Improving sign language translation with monolingual data by sign back-translation. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition; 2021. p. 1316–1325.
5. Camgoz NC, Koller O, Hadfield S, Bowden R. Multi-channel transformers for multi-articulatory sign language translation. In: *Computer Vision—ECCV 2020 Workshops: Glasgow, UK, August 23–28, 2020, Proceedings, Part IV* 16. Springer; 2020. p. 301–319.
6. Camgöz NC, Saunders B, Rochette G, Giovanelli M, Inches G, Nachtrab-Ribback R, et al. Content4all open research sign language translation datasets. In: 2021 16th IEEE International Conference on Automatic Face and Gesture Recognition (FG 2021). IEEE; 2021. p. 1–5.
7. Orbay A, Akarun L. Neural sign language translation by learning tokenization. In: 2020 15th IEEE International Conference on Automatic Face and Gesture Recognition (FG 2020). IEEE; 2020. p. 222–228.
8. Shi B, Brentari D, Shakhnarovich G, Livescu K. Open-domain sign language translation learned from online video. *arXiv preprint arXiv:220512870*. 2022;.
9. Yin K, Read J. Better sign language translation with STMC-transformer. *arXiv preprint arXiv:200400588*. 2020;.
10. Cheng KL, Yang Z, Chen Q, Tai YW. Fully convolutional networks for continuous sign language recognition. In: *Computer Vision—ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XXIV* 16. Springer; 2020. p. 697–714.
11. Cihan Camgoz N, Hadfield S, Koller O, Bowden R. Subunets: End-to-end hand shape and continuous sign language recognition. In: Proceedings of the IEEE international conference on computer vision; 2017. p. 3056–3065.
12. Cui R, Liu H, Zhang C. Recurrent convolutional neural networks for continuous sign language recognition by staged optimization. In: Proceedings of the IEEE conference on computer vision and pattern recognition; 2017. p. 7361–7369.
13. Cui R, Liu H, Zhang C. A deep neural framework for continuous sign language recognition by iterative training. *IEEE Transactions on Multimedia*. 2019; p. 1880–1891. <https://doi.org/10.1109/TMM.2018.2889563>
14. Huang J, Zhou W, Zhang Q, Li H, Li W. Video-based sign language recognition without temporal segmentation. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 32; 2018.
15. Min Y, Hao A, Chai X, Chen X. Visual alignment constraint for continuous sign language recognition. In: Proceedings of the IEEE/CVF International Conference on Computer Vision; 2021. p. 11542–11551.
16. Zhou H, Zhou W, Zhou Y, Li H. Spatial-temporal multi-cue network for continuous sign language recognition. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 34; 2020. p. 13009–13016.
17. Koller O, Zargaran O, Ney H, Bowden R. Deep sign: Hybrid CNN-HMM for continuous sign language recognition. In: Proceedings of the British Machine Vision Conference 2016; 2016.
18. Dreuw P, Rybach D, Deselaers T, Zahedi M, Ney H. Speech recognition techniques for a sign language recognition system. *hand*. 2007; 60:80.
19. Ong SC, Ranganath S. Automatic sign language analysis: A survey and the future beyond lexical meaning. *IEEE Transactions on Pattern Analysis & Machine Intelligence*. 2005; 27(06):873–891. <https://doi.org/10.1109/TPAMI.2005.112> PMID: 15943420
20. Das S, Dai R, Yang D, Bremond F. Vpn++: Rethinking video-pose embeddings for understanding activities of daily living. *IEEE Transactions on Pattern Analysis and Machine Intelligence*. 2021; 44(12):9703–9717. <https://doi.org/10.1109/TPAMI.2021.3127885>

21. Duan H, Zhao Y, Chen K, Lin D, Dai B. Revisiting skeleton-based action recognition. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition; 2022. p. 2969–2978.
22. Bruce X, Liu Y, Zhang X, Zhong Sh, Chan KC. Mmnet: A model-based multimodal network for human action recognition in rgb-d videos. *IEEE Transactions on Pattern Analysis and Machine Intelligence*. 2022; 45(3):3522–3538.
23. Huang J, Zhou W, Li H, Li W. Sign language recognition using 3d convolutional neural networks. In: 2015 IEEE international conference on multimedia and expo (ICME). IEEE; 2015. p. 1–6.
24. Li D, Rodriguez C, Yu X, Li H. Word-level deep sign language recognition from video: A new large-scale dataset and methods comparison. In: Proceedings of the IEEE/CVF winter conference on applications of computer vision; 2020. p. 1459–1469.
25. Li D, Yu X, Xu C, Petersson L, Li H. Transferring cross-domain knowledge for video sign language recognition. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition; 2020. p. 6205–6214.
26. Martínez AM, Wilbur RB, Shay R, Kak AC. Purdue RVL-SLLL ASL database for automatic recognition of American Sign Language. In: Proceedings. Fourth IEEE International Conference on Multimodal Interfaces. IEEE; 2002. p. 167–172.
27. Starner T, Weaver J, Pentland A. Real-time american sign language recognition using desk and wearable computer based video. *IEEE Transactions on pattern analysis and machine intelligence*. 1998; 20(12):1371–1375. <https://doi.org/10.1109/34.735811>
28. Jozse HRV, Koller O. Ms-asl: A large-scale data set and benchmark for understanding american sign language. *arXiv preprint arXiv:181201053*. 2018;.
29. Duarte A, Palaskar S, Ventura L, Ghadiyaram D, DeHaan K, Metze F, et al. How2sign: a large-scale multimodal dataset for continuous american sign language. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition; 2021. p. 2735–2744.
30. Dhruvo SE, Rahman MA, Mandal MK, Shihab MIH, Ansary A, Shithi KF, et al. Bornil: An open-source sign language data crowdsourcing platform for AI enabled dialect-agnostic communication. *arXiv preprint arXiv:230815402*. 2023;.
31. Tarrés L, Gállego GI, Duarte A, Torres J, Giró-i Nieto X. Sign Language Translation from Instructional Videos. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition; 2023. p. 5624–5634.
32. Nihal RA, Rahman S, Broti NM, Deowan SA. Bangla sign alphabet recognition with zero-shot and transfer learning. *Pattern Recognition Letters*. 2021; 150:84–93. <https://doi.org/10.1016/j.patrec.2021.06.020>
33. Sutskever I, Vinyals O, Le QV. Sequence to sequence learning with neural networks. *Advances in neural information processing systems*. 2014; 27.
34. Guo D, Zhou W, Li H, Wang M. Hierarchical LSTM for sign language translation. In: Proceedings of the AAAI conference on artificial intelligence. vol. 32; 2018.
35. Kim Y, Kwak M, Lee D, Kim Y, Baek H. Keypoint based sign language translation without glosses. *arXiv preprint arXiv:220410511*. 2022;.
36. Ko SK, Kim CJ, Jung H, Cho C. Neural sign language translation based on human keypoint estimation. *Applied sciences*. 2019; 9(13):2683. <https://doi.org/10.3390/app9132683>
37. Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, et al. Attention is all you need. In: *Advances in neural information processing systems*. vol. 30; 2017.
38. Li H, Jiang H, Jin T, Li M, Chen Y, Lin Z, et al. DATE: Domain Adaptive Product Seeker for E-commerce. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition; 2023. p. 19315–19324.
39. Huang R, Ren Y, Liu J, Cui C, Zhao Z. Generspeech: Towards style transfer for generalizable out-of-domain text-to-speech. *Advances in Neural Information Processing Systems*. 2022; 35:10970–10983.
40. Huang R, Zhao Z, Liu H, Liu J, Cui C, Ren Y. Prodiff: Progressive fast diffusion model for high-quality text-to-speech. In: Proceedings of the 30th ACM International Conference on Multimedia; 2022. p. 2595–2605.
41. Huang R, Liu J, Liu H, Ren Y, Zhang L, He J, et al. Transpeech: Speech-to-speech translation with bilateral perturbation. *arXiv preprint arXiv:220512523*. 2022;.
42. Lin Z, Zhao Z, Li H, Liu J, Zhang M, Zeng X, et al. Simullr: Simultaneous lip reading transducer with attention-guided adaptive memory. In: Proceedings of the 29th ACM International Conference on Multimedia; 2021. p. 1359–1367.
43. Xia Y, Zhao Z, Ye S, Zhao Y, Li H, Ren Y. Video-guided curriculum learning for spoken video grounding. In: Proceedings of the 30th ACM International Conference on Multimedia; 2022. p. 5191–5200.

44. Liu J, Wang X, Wang C, Gao Y, Liu M. Temporal decoupling graph convolutional network for skeleton-based gesture recognition. *IEEE Transactions on Multimedia*. 2023; 26: 811–823.
45. Lu Y, Zhu Y, Lu G. 3d sceneflownet: Self-supervised 3d scene flow estimation based on graph cnn. In: 2021 IEEE International Conference on Image Processing (ICIP). IEEE; 2021. p. 3647–3651.
46. Correia de Amorim C, Macêdo D, Zanchettin C. Spatial-temporal graph convolutional networks for sign language recognition. *arXiv e-prints*. 2019; p. arXiv–1901.
47. Tunga A, Nuthalapati SV, Wachs J. Pose-based sign language recognition using GCN and BERT. In: Proceedings of the IEEE/CVF winter conference on applications of computer vision; 2021. p. 31–40.
48. Vadiš Q, Carreira J, Zisserman A. Action Recognition? A New Model and the Kinetics Dataset. Joao Carreira, Andrew Zisserman;.
49. Albanie S, Varol G, Momeni L, Afouras T, Chung JS, Fox N, et al. BSL-1K: Scaling up co-articulated sign language recognition using mouthing cues. In: Computer Vision—ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XI 16. Springer; 2020. p. 35–53.
50. Dey S, Pal A, Chaabani C, Koller O. Clean Text and Full-Body Transformer: Microsoft's Submission to the WMT22 Shared Task on Sign Language Translation. *arXiv preprint arXiv:221013326*. 2022;.
51. Duarte A, Albanie S, Giró-i Nieto X, Varol G. Sign language video retrieval with free-form textual queries. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition; 2022. p. 14094–14104.
52. Li D, Xu C, Yu X, Zhang K, Swift B, Suominen H, et al. Tspnet: Hierarchical feature learning via temporal semantic pyramid for sign language translation. In: Advances in Neural Information Processing Systems. vol. 33; 2020. p. 12034–12045.
53. Xie S, Sun C, Huang J, Tu Z, Murphy K. Rethinking spatiotemporal feature learning for video understanding. *arXiv preprint arXiv:171204851*. 2017;1(2):5.
54. Kay W, Carreira J, Simonyan K, Zhang B, Hillier C, Vijayanarasimhan S, et al. The kinetics human action video dataset. *arXiv preprint arXiv:170506950*. 2017;.
55. Cao Z, Hidalgo G, Simon T, Wei S, Sheikh Y. OpenPose: Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields. *IEEE Transactions on Pattern Analysis & Machine Intelligence*. 2021; 43(01):172–186. <https://doi.org/10.1109/TPAMI.2019.2929257> PMID: 31331883
56. Lugaresi C, Tang J, Nash H, McClanahan C, Uboweja E, Hays M, et al. Mediapipe: A framework for building perception pipelines. *arXiv preprint arXiv:190608172*. 2019;.
57. Müller M, Ebling S, Avramidis E, Battisti A, Berger M, Bowden R, et al. Findings of the first wmt shared task on sign language translation (wmt-slt22). In: Proceedings of the Seventh Conference on Machine Translation (WMT); 2022. p. 744–772.
58. Gan S, Yin Y, Jiang Z, Xie L, Lu S. Skeleton-aware neural sign language translation. In: Proceedings of the 29th ACM International Conference on Multimedia; 2021. p. 4353–4361.
59. Carreira J, Zisserman A. Quo vadis, action recognition? a new model and the kinetics dataset. In: proceedings of the IEEE Conference on Computer Vision and Pattern Recognition; 2017. p. 6299–6308.
60. Kipf TN, Welling M. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:160902907*. 2016;.
61. Kudo T, Richardson J. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. *arXiv preprint arXiv:180806226*. 2018;.
62. Zhou B, Chen Z, Clapés A, Wan J, Liang Y, Escalera S, et al. Gloss-free sign language translation: Improving from visual-language pretraining. In: Proceedings of the IEEE/CVF International Conference on Computer Vision; 2023. p. 20871–20881.
63. Yin A, Zhong T, Tang L, Jin W, Jin T, Zhao Z. Gloss attention for gloss-free sign language translation. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition; 2023. p. 2551–2562.
64. Tang S, Guo D, Hong R, Wang M. Graph-based multimodal sequential embedding for sign language translation. *IEEE Transactions on Multimedia*. 2021; 24:4433–4445. <https://doi.org/10.1109/TMM.2021.3117124>
65. Xu W, Ying J, Yang H, Liu J, Hu X. Residual spatial graph convolution and temporal sequence attention network for sign language translation. *Multimedia Tools and Applications*. 2023; 82(15):23483–23507. <https://doi.org/10.1007/s11042-022-14172-5>
66. Vázquez-Enríquez M, Alba-Castro JL, Docío-Fernández L, Rodríguez-Banga E. Isolated sign language recognition with multi-scale spatial-temporal graph convolutional networks. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition; 2021. p. 3462–3471.

67. Naz N, Sajid H, Ali S, Hasan O, Ehsan MK. Signgraph: An efficient and accurate pose-based graph convolution approach toward sign language recognition. *IEEE Access*. 2023; 11:19135–19147. <https://doi.org/10.1109/ACCESS.2023.3247761>
68. Miah ASM, Hasan MAM, Nishimura S, Shin J. Sign language recognition using graph and general deep neural network based on large scale dataset. *IEEE Access*. 2024;.
69. Kan J, Hu K, Hagenbuchner M, Tsoi AC, Bennamoun M, Wang Z. Sign language translation with hierarchical spatio-temporal graph neural network. In: *Proceedings of the IEEE/CVF winter conference on applications of computer vision*; 2022. p. 3367–3376.
70. Papineni K, Roukos S, Ward T, Zhu WJ. BLEU: A Method for Automatic Evaluation of Machine Translation. In: *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics*. Association for Computational Linguistics; 2002. p. 311–318.
71. Roy P, Han JE, Chouhan S, Thumu B. American Sign Language Video to Text Translation. *arXiv preprint arXiv:240207255*. 2024;.
72. Aoxiong Y, Tianyun Z, Li T, Weike J, Tao J, Zhou Z. Gloss Attention for Gloss-free Sign Language Translation. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE; 2023.
73. Radford A, Kim JW, Hallacy C, Ramesh A, Goh G, Agarwal S, et al. Learning transferable visual models from natural language supervision. In: *International conference on machine learning*. PMLR; 2021. p. 8748–8763.
74. Luong MT, Pham H, Manning CD. Effective approaches to attention-based neural machine translation. *arXiv preprint arXiv:150804025*. 2015;.
75. Bahdanau D, Cho K, Bengio Y. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:14090473*. 2014;.